



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Análisis y diseño de sistema 2

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD #:4

Requerimientos no funcionales

Anuncios Importantes



Anuncio 1 (Ingresa los anuncios importantes como lo sean, cortos, entregas, practicas, proyectos, horarios de calificación, etc.)



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



Dudas del proyecto



Teoría



Caso práctico

COMPETENCIA(S) QUE DESARROLLAREMOS



Diseña arquitecturas de software mediante la definición de componentes, conexiones y capas del sistema para asegurar la funcionalidad, rendimiento y escalabilidad del producto final



VALOR DE LA SEMANA

El liderazgo es la capacidad de influir, guiar y motivar a un grupo de personas para alcanzar objetivos comunes, promoviendo colaboración, compromiso y desarrollo del equipo.

CONTENIDO

01

pruebas no
funcioanles

02

pruebas de carga

03

pruebas de estres

04

pruebas de
escalabilidad

05

pruebas de
portabilidad





PRUEBAS

NO

FUNCIONALES



¿QUE SON LAS PRUEBAS NO FUNCIONALES?



Las pruebas no funcionales son un tipo de pruebas de software que se realizan para verificar los aspectos no funcionales de una aplicación de software. En lugar de centrarse en lo que el software "hace" (sus funcionalidades), estas pruebas se enfocan en cómo lo hace, es decir, en su rendimiento, usabilidad, fiabilidad, seguridad y escalabilidad, entre otros atributos de calidad.



Imagina un coche. Las pruebas funcionales se asegurarían de que el motor enciende y las ruedas giran. Las pruebas no funcionales verificarían qué tan rápido puede ir, qué tan cómodo es el asiento y cuánto combustible consume.



Puntos clave de las pruebas no funcionales

Una aplicación puede tener todas sus funciones correctas, pero si es lenta o difícil de usar, los usuarios se frustrarán y la abandonarán. Las pruebas no funcionales, como las de usabilidad, garantizan que la experiencia sea fluida y positiva.

Dentro de las pruebas no funcionales, hay varias categorías, como las pruebas de rendimiento (carga, estrés, volumen), de seguridad, de usabilidad, de portabilidad y de escalabilidad. Cada una de ellas se centra en un aspecto específico de la calidad del software.

Evalúan la calidad y el rendimiento

Garantizan la experiencia del usuario

Reducen riesgos

Tipos comunes

Estas pruebas no se centran en si una característica funciona o no, sino en su rendimiento. Por ejemplo, una prueba funcional podría verificar que un botón de inicio de sesión funciona, mientras que una prueba no funcional de carga evaluaría si ese botón sigue funcionando correctamente cuando 1000 usuarios intentan iniciar sesión al mismo tiempo.

Identificar problemas de rendimiento o seguridad en las primeras etapas del desarrollo puede prevenir fallos mayores en producción. Un sitio web que no soporta un gran número de visitantes podría colapsar durante un evento importante, causando pérdidas económicas y de reputación. Las pruebas de estrés y de carga ayudan a prevenir estos escenarios.

Principios Clave

Los resultados de las pruebas solo son válidos si se ejecutan en un entorno que simula fielmente el entorno de producción. Si las pruebas se realizan en un entorno de desarrollo con pocos recursos, los resultados no serán representativos de lo que ocurrirá en el entorno de producción.

Los objetivos de las pruebas no funcionales deben estar alineados con las expectativas del negocio. Por ejemplo, una aplicación para un evento masivo necesita un enfoque en la escalabilidad, mientras que un sistema bancario requiere un enfoque en la seguridad.

Medición y Métricas

**Pruebas en Entornos
Realistas**

**Evaluación
Temprana y Continua**

**Tipos
comunes**

Las pruebas no funcionales se basan en datos cuantificables. No es suficiente decir que un sistema es "rápido"; es necesario definir una métrica, como "el tiempo de respuesta de la página de inicio de sesión no debe exceder los 2 segundos".

Los problemas no funcionales son a menudo difíciles y costosos de solucionar si se descubren tarde. Por ello, es crucial empezar a considerar y planificar estas pruebas desde las primeras etapas del ciclo de vida del desarrollo de software.

Importancia dentro de una arquitectura de software

- La importancia de las pruebas no funcionales dentro de una arquitectura de software es crucial, ya que se enfocan en los atributos de calidad que determinan si el sistema será exitoso en el mundo real, más allá de que sus funcionalidades básicas operen correctamente. Su aplicabilidad es transversal y debe ser considerada desde las fases más tempranas del diseño arquitectónico.



- Importancia
- Satisfacción del usuario final: Un software que cumple todas sus funciones, pero es lento o inestable, genera una mala experiencia de usuario. Las pruebas no funcionales aseguran que la aplicación sea rápida, fiable y fácil de usar, lo cual es clave para la retención de clientes y la reputación de la marca.
- Reducción de riesgos: Identificar problemas de rendimiento o vulnerabilidades de seguridad en las etapas iniciales del desarrollo es mucho menos costoso que solucionarlos una vez que el sistema ya está en producción. Un fallo de seguridad o una caída del sistema pueden resultar en grandes pérdidas financieras y de datos.
- Optimización de recursos: Permiten identificar cuellos de botella y problemas de rendimiento que podrían requerir una reestructuración de la arquitectura. Al hacerlo de forma anticipada, se puede optimizar el uso de hardware y software, evitando gastos innecesarios.
- Cumplimiento de requisitos: Muchos requisitos del negocio, aunque no se consideren "funcionales" directamente, son críticos para la operación. Por ejemplo, un requisito de negocio podría ser que "el sitio web debe ser capaz de manejar 10.000 usuarios simultáneos en un evento de venta". Las pruebas no funcionales son las únicas que pueden verificar este

Aplicabilidad en la Arquitectura de Software

- **Diseño arquitectónico:** Durante el diseño de la arquitectura, se deben considerar los requisitos no funcionales. Por ejemplo, si se sabe que la aplicación necesitará manejar un gran volumen de tráfico, la arquitectura debe diseñarse para ser escalable, quizás utilizando microservicios o una arquitectura en la nube. Las pruebas no funcionales validarán que esas decisiones de diseño sean efectivas.
- **Selección de tecnologías:** La elección de bases de datos, lenguajes de programación, frameworks y servidores debe basarse, en parte, en los requisitos no funcionales. Si se necesita una alta disponibilidad, se seleccionarán tecnologías que la ofrezcan. Las pruebas de fiabilidad confirmarán que las tecnologías elegidas cumplen con estas expectativas.
- **Entornos de prueba:** La aplicabilidad de las pruebas no funcionales requiere que se realicen en un entorno que simule lo más fielmente posible el entorno de producción. Un entorno de prueba con recursos limitados no proporcionará resultados fiables sobre el rendimiento bajo una carga de trabajo real.
- **Monitoreo continuo:** Una vez que el software está en producción, la aplicabilidad de las pruebas no funcionales continúa a través del monitoreo. Las herramientas de monitoreo en tiempo real ayudan a detectar desviaciones en el rendimiento, lo que puede indicar la necesidad de optimizaciones o de un nuevo ciclo de pruebas.



PRUEBAS DE CARGA



A background image showing an office environment. On the left, a man in a light blue shirt is partially visible. In the center, two women are looking at a screen; one wears glasses and a patterned scarf. On the right, a woman with curly hair is smiling. The office has a modern feel with glass partitions and a mezzanine level with chairs in the background.

PRUEBAS DE CARGA

son un tipo de pruebas de rendimiento que se realizan para evaluar el comportamiento de un sistema de software bajo una carga de trabajo anticipada y realista. El objetivo principal es determinar si la aplicación puede manejar la cantidad de usuarios y transacciones esperadas sin degradar su rendimiento.

¿QUÉ EVALÚAN LAS PRUEBAS DE CARGA?



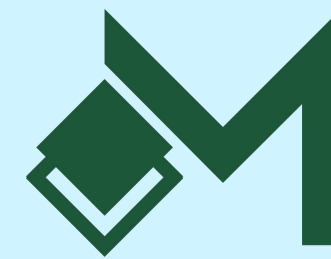
Tiempo de respuesta

Miden el tiempo que tarda el sistema en responder a las solicitudes de los usuarios. Un tiempo de respuesta rápido es crucial para una buena experiencia de usuario.



Utilización de recursos

Evalúan cómo la aplicación utiliza recursos del sistema como la CPU, la memoria, la red y la base de datos bajo una carga normal. Esto ayuda a identificar cuellos de botella y posibles problemas de rendimiento.



Comportamiento o del sistema

Aseguran que el sistema no se congele, se bloquee o muestre errores bajo la carga esperada.

¿Por qué son importantes?

Imagina una tienda en línea que se prepara para una venta de "Black Friday". Saben que esperan 10,000 visitantes simultáneos. Si el sistema no se prueba para soportar esa carga, podría volverse increíblemente lento o colapsar, resultando en pérdida de ventas e insatisfacción del cliente. Las pruebas de carga ayudan a prevenir estos escenarios al simular el tráfico esperado de antemano y asegurar que la arquitectura de software y el hardware sean lo suficientemente robustos.

¿Cómo se realizan?

Se diseñan scripts que imitan el comportamiento de los usuarios reales. Por ejemplo, un script podría simular a un usuario navegando por productos, agregando artículos al carrito y finalizando una compra.

Se analizan los datos recopilados para identificar problemas de rendimiento y se ajusta el sistema según sea necesario.

Definir los objetivos

**Crear el
escenario**

**Ejecutar la
prueba**

**Analizar los
resultados**

Se establecen métricas claras como el tiempo de respuesta máximo aceptable y el número de usuarios concurrentes a simular.

Se utilizan herramientas de automatización de pruebas de carga (como Apache JMeter o LoadRunner) para generar un gran número de usuarios virtuales que ejecutan los scripts de forma simultánea.



EJEMPLO PRACTICO



PRUEBAS STRESS



A background image showing an office environment. In the foreground, a man in a light blue shirt is partially visible on the left, and a woman with curly hair is on the right. In the background, two women are looking at a screen. The image is overlaid with a dark blue semi-transparent layer.

PRUEBAS DE STRESS

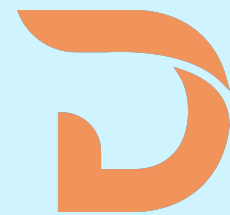
son un tipo de pruebas de rendimiento que se realizan para evaluar el comportamiento de un sistema de software bajo una carga de trabajo extrema, más allá de sus límites normales. El objetivo no es solo ver si el sistema se mantiene, sino también cómo falla y si se recupera de manera adecuada.

¿Qué evalúan las pruebas de estrés?



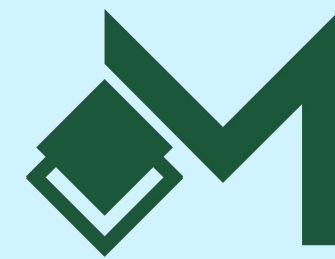
Punto de quiebre

Su principal objetivo es encontrar el punto exacto en el que el sistema se rompe o deja de funcionar de forma aceptable.



Comportamiento o bajo presión

Evalúan cómo el sistema maneja la sobrecarga de usuarios o transacciones. ¿Se congela, se bloquea o simplemente se vuelve más lento?



Capacidad de recuperación:

Una vez que el estrés desaparece, ¿el sistema se recupera de forma automática y vuelve a su estado normal? O, por el contrario, ¿requiere una intervención manual para reiniciarse?

¿Por qué son importantes?

Piensa en un sistema de compra de boletos para un concierto muy popular. Durante la venta, miles de personas intentarán comprar boletos en cuestión de segundos. Esto creará un pico de tráfico que es mucho más alto que el tráfico promedio. Si el sistema no ha sido probado para este escenario, podría colapsar por completo, causando frustración masiva, pérdida de ventas y daños a la reputación de la empresa. Las pruebas de estrés ayudan a prepararse para estos eventos de alto impacto.

¿Cómo se realizan?

Similar a las pruebas de carga, se usan herramientas de automatización como k6, JMeter o LoadRunner para generar un tráfico artificial.

Durante la prueba, se monitorean de cerca los recursos del servidor (CPU, memoria, base de datos) para entender qué componente está causando el cuello de botella.

Definir el límite

Crear el escenario

Aumentar la carga de forma gradual

Monitorear y analizar

Evaluar la recuperación

Se establece un número de usuarios o un volumen de transacciones que es significativamente mayor al esperado.

A menudo, la carga se incrementa de forma progresiva hasta que el sistema alcanza su punto de saturación y empieza a mostrar errores o tiempos de respuesta inaceptables.

Después de la prueba de estrés, se reduce la carga a niveles normales para ver si el sistema se recupera por sí solo.

En resumen, mientras las pruebas de carga confirman que un sistema puede manejar lo que se espera, las pruebas de estrés lo preparan para lo inesperado.



EJEMPLO PRACTICO



PRUEBAS DE **ESCALABILIDAD**





PRUEBAS DE ESCALABILIDAD

son un tipo de pruebas de rendimiento que se realizan para determinar la capacidad de un sistema, una red o una aplicación para manejar un aumento en el volumen de trabajo de manera eficiente. Su objetivo es asegurar que el sistema puede crecer en términos de usuarios, transacciones o datos sin que su rendimiento se degrade.

¿Por qué son importantes?

Imagina una red social o una plataforma de comercio electrónico. Si la base de usuarios crece exponencialmente, la plataforma debe ser capaz de soportar a todos esos nuevos usuarios sin volverse lenta o inestable. Las pruebas de escalabilidad garantizan que la arquitectura de software y la infraestructura subyacente puedan soportar este crecimiento futuro. Sin ellas, una empresa podría invertir en hardware costoso que no logra el rendimiento deseado o, peor aún, su plataforma podría colapsar cuando más lo necesita.

¿Qué evalúan las pruebas de escalabilidad?



Capacidad de crecimiento

Miden cómo responde el sistema a un incremento gradual en la carga. Se busca identificar el punto en el que el rendimiento empieza a caer, incluso cuando se añaden más recursos (como servidores o memoria).



Eficiencia de los recursos

Evalúan si el sistema utiliza los recursos de manera eficiente a medida que la carga aumenta. Por ejemplo, ¿duplicar los servidores realmente duplica la capacidad?



Cuellos de botella

Ayudan a identificar componentes (como la base de datos, la red o un servidor específico) que impiden que el sistema escale correctamente.

¿Cómo se realizan?

La prueba se inicia con una carga baja y se va aumentando de manera constante y controlada. Esto puede hacerse añadiendo más usuarios virtuales o más transacciones por segundo.

Se observa el punto en el que el rendimiento comienza a degradarse. Este es el punto de inflexión de la escalabilidad, más allá del cual el sistema ya no puede crecer de manera eficiente.

Definir el límite
Incrementar
la carga

Monitorear el
rendimiento

Identificar el punto
de inflexión

Analizar y optimizar

Se establecen métricas de rendimiento y se define el objetivo de escalabilidad. Por ejemplo, "el sistema debe soportar 1 millón de usuarios registrados sin que el tiempo de respuesta de la página de inicio de sesión exceda los 500 ms".

Durante todo el proceso, se monitorean las métricas clave de rendimiento (como el tiempo de respuesta y la utilización de recursos del servidor) para ver cómo se comportan a medida que la carga aumenta.

Con la información obtenida, se pueden tomar decisiones informadas sobre cómo optimizar la arquitectura para mejorar la escalabilidad, como optimizar consultas a la base de datos, implementar balanceadores de carga o reestructurar el código.



EJEMPLO PRACTICO



PRUEBAS DE **PORTABILIDAD**



A background image showing an office environment. On the left, a man in a light blue shirt is partially visible. In the center, two women are looking at a screen; one is wearing glasses and a patterned scarf. On the right, a woman with curly hair is smiling. The office has glass partitions and a modern interior.

PRUEBAS DE PORTABILIDAD

son un tipo de pruebas no funcionales que se realizan para verificar si un sistema de software puede ser transferido de un entorno a otro sin problemas. El objetivo es asegurar que la aplicación funcione de manera consistente y correcta en diferentes sistemas operativos, navegadores web, dispositivos móviles y configuraciones de hardware.

¿Qué evalúan las pruebas de portabilidad?



Compatibilidad con el entorno

Miden qué tan bien el software se adapta a nuevos entornos. Esto incluye la instalación, ejecución y uso en diferentes plataformas.



Consistencia de la interfaz

Verifican que la interfaz de usuario (UI) se muestre de manera correcta y sin errores visuales en distintas resoluciones de pantalla, tamaños de ventana o navegadores.



Integración de datos

Aseguran que los datos puedan ser transferidos y utilizados correctamente entre diferentes sistemas, si es necesario

¿Por qué son importantes?

En un mundo donde los usuarios acceden a la misma aplicación desde múltiples dispositivos (un ordenador con Windows, una Mac, un teléfono Android o una tablet con iOS), la portabilidad es fundamental. Si una aplicación web no se muestra correctamente en Safari o una aplicación de escritorio no funciona en Linux, se puede perder una porción significativa de la base de usuarios. Las pruebas de portabilidad garantizan una experiencia de usuario uniforme y amplía el alcance del software a un mercado más grande.

¿Cómo se realizan?

Se configuran los dispositivos o máquinas virtuales con los sistemas operativos, navegadores y otras configuraciones de software necesarias para la prueba.

Se documentan los errores de compatibilidad y se reportan al equipo de desarrollo para que se corrijan. Los errores comunes incluyen problemas de diseño (la UI se rompe en un navegador), fallos de rendimiento o incompatibilidades de funciones específicas.

Definir los entornos

Preparar los entornos

Ejecutar las pruebas

Reportar y corregir

Se identifican y se priorizan los entornos de destino. Por ejemplo, "probar la aplicación en Chrome, Firefox y Safari en Windows 10, macOS y las últimas versiones de iOS y Android

Se ejecutan los casos de prueba funcionales y no funcionales en cada uno de los entornos. El objetivo es verificar que todo funciona exactamente igual que en el entorno de desarrollo.

Tipo de Prueba	Propósito	Objetivo Principal	Cuándo se Realiza
Carga	Evaluar el comportamiento del sistema bajo una carga de trabajo normal y esperada.	Verificar que el sistema puede manejar el número de usuarios y transacciones previstos sin degradar el rendimiento.	Durante el desarrollo, antes del lanzamiento, para validar el rendimiento.
Estrés	Evaluar cómo se comporta el sistema bajo una carga de trabajo extrema, más allá de sus límites.	Encontrar el punto de quiebre del sistema y probar su capacidad de recuperación.	Después de las pruebas de carga, para preparar el sistema ante picos inesperados de tráfico.
Escalabilidad	Determinar la capacidad del sistema para crecer y manejar un aumento en la carga de trabajo.	Identificar si la arquitectura puede soportar un crecimiento futuro sin comprometer el rendimiento.	Durante el diseño de la arquitectura y el desarrollo, de manera iterativa.
Portabilidad	Verificar que la aplicación funciona correctamente en diferentes entornos de hardware y software.	Asegurar una experiencia de usuario consistente en múltiples sistemas operativos, navegadores y dispositivos.	Durante las fases de desarrollo y pruebas de regresión.







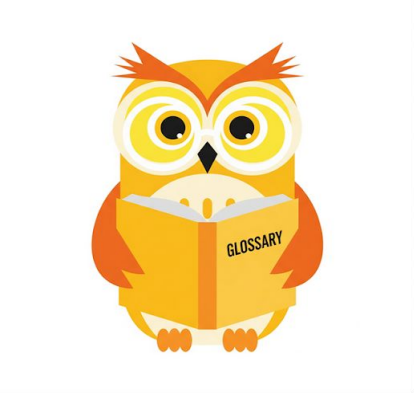
Ejemplo práctico

Contexto: Imaginad que vuestro sistema (el de la cafetería o el de noticias) es crítico. Si el sistema se cae, la empresa pierde dinero cada minuto. No basta con decir "que no se caiga", hay que definir un **EAC medible**.

1. El Enunciado del Ejemplo (Lo que tú les das)

"El sistema de procesamiento de pedidos debe tener una **disponibilidad del 99.9% (Triple Nueve)** durante el horario de operación comercial (08:00 a 20:00), con un tiempo de recuperación (MTTR) no mayor a **5 minutos** en caso de fallo crítico."

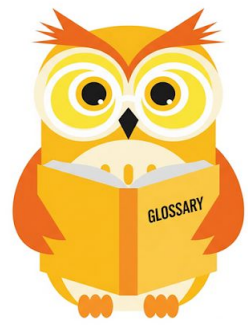




Practiquemos lo aprendido

1. Investigad y completad la siguiente tabla basada en el estándar **OWASP Top 10**. Para cada vulnerabilidad, debéis proponer una medida de contingencia técnica (prevención).

#	Vulnerabilidad (Nombre)	Descripción Breve	Medida de Contingencia
1	Inyección (SQL, NoSQL)	Datos no confiables enviados al intérprete.	Uso de consultas preparadas (Prepared Statements).
2	Pérdida de Autenticación	Fallos en la gestión de sesiones o contraseñas.	Implementar MFA (Autenticación de Dos Factores).
3	Exposición de Datos Sensibles	Falta de cifrado en datos en reposo o tránsito.	Cifrado AES-256 y uso de protocolos TLS.
4	XML External Entities (XXE)	Procesamiento de entidades externas en XML.	Deshabilitar el procesamiento de entidades externas.
5	Control de Acceso Quebrado	Usuarios acceden a funciones fuera de su permiso.	Implementar el principio de "mínimo privilegio".
6	Configuración de Seguridad Incorrecta	Ajustes por defecto o servicios innecesarios.	Endurecimiento (Hardening) de servidores y apps.
7	Cross-Site Scripting (XSS)	Inyección de scripts en páginas vistas por otros.	Escapar datos de salida y validar entradas.
8	Deserialización Insegura	Ejecución de código al convertir datos en objetos.	No aceptar objetos serializados de fuentes dudosas.
9	Componentes Vulnerables	Uso de librerías antiguas con fallos conocidos.	Auditoría de dependencias y parches constantes.
10	Registro y Monitoreo Insuficiente	No detectar ataques en tiempo real.	Centralización de logs y alertas automáticas.



Practiquemos lo aprendido

Elege **una** de las vulnerabilidades anteriores y redacta un **Escenario de Atributo de Calidad (EAC)** formal. Recuerda que un EAC debe tener:

1. **Fuente del estímulo:** ¿Quién ataca? (ej. Un hacker externo).
2. **Estímulo:** ¿Qué intenta hacer? (ej. Inyectar código SQL).
3. **Artefacto:** ¿Qué parte del sistema es atacada? (ej. La base de datos de usuarios).
4. **Respuesta:** ¿Qué hace el sistema? (ej. El sistema bloquea la petición y registra el evento).
5. **Medida de respuesta:** ¿Cómo lo medimos? (ej. El ataque debe ser neutralizado en menos de 1 segundo).



MUCHAS GRACIAS